FPAI DEMO说明

一. 工具链

FPGA工程仅适用于Vivado2018.03。

二. FPGA工程源文件

```
── constrs_1
  |── ai7030.xdc                 // 约束文件
  |── impl_constraints.xdc       // 约束文件
── PS_AI IP
  |── processing_system7_v5_5    // PS IP
── rtl
  |── adder_op                   // 示例custom op
  |── edif                       // AI_Mate网表文件
  |── ai7030_edif_top.v          // 顶层模块
  |── AI_Mate_stub.v             // AI_Mate模块
  |── sdi                        // 包含rx_sdi模块
  |── ....
```

edif文件夹中，默认使用的是带DetPost硬算子的网表，名为AI_Mate.edif。若不需要使用DetPost硬算子，可以使用AI_Mate_no_detpost.edif替代AI_Mate.edif，降低PL逻辑资源的占用率。

根据使用的开发板型号，修改impl_constraints.xdc文件的管脚约束。

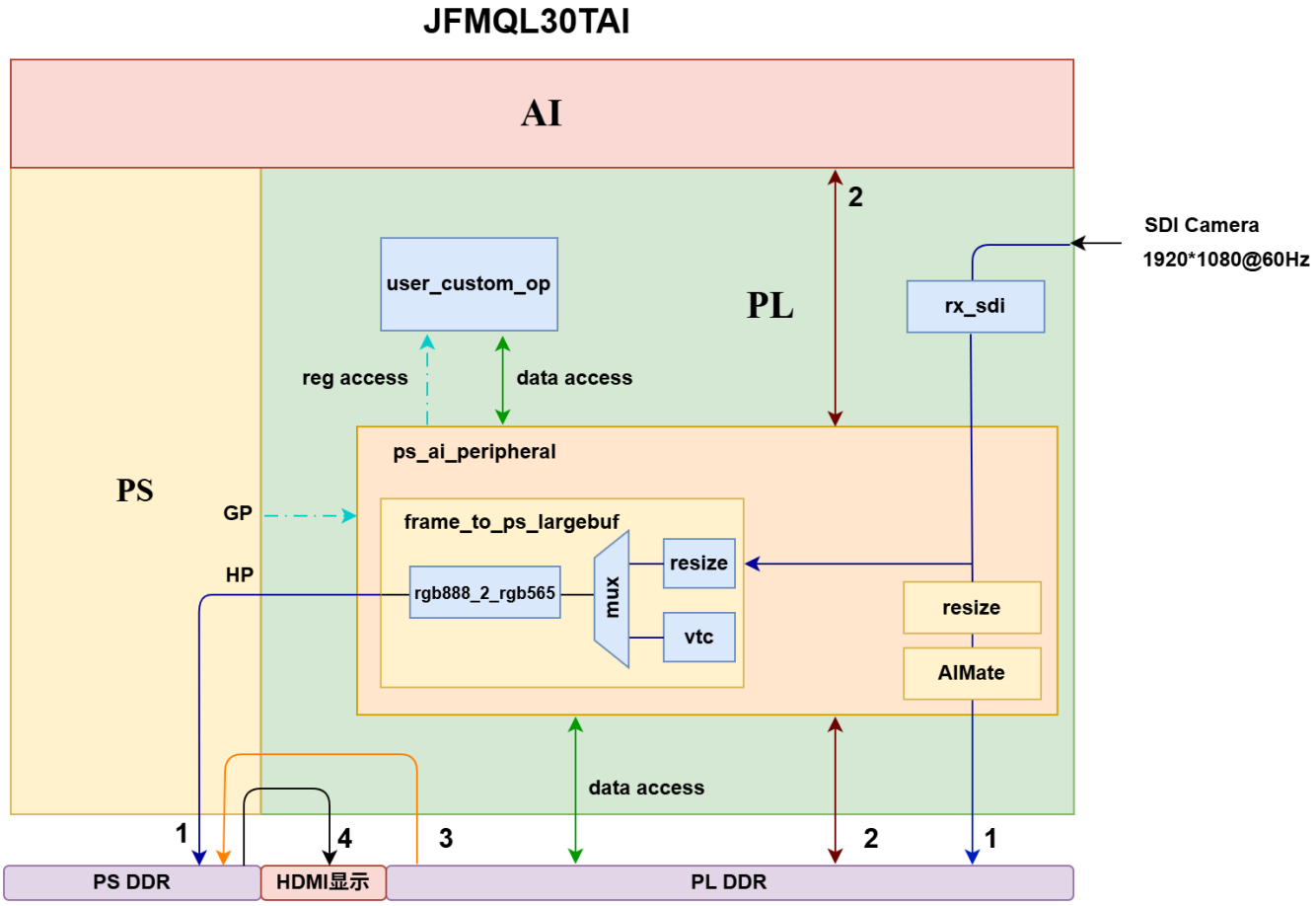
（1）悟净开发板

```
set_property PACKAGE_PIN U6 [get_ports FMC_HPC_GBTCLK0_M2C_C_P]  ## WUJING BOARD
set_property PACKAGE_PIN U5 [get_ports FMC_HPC_GBTCLK0_M2C_C_N]
```

（2）悟净开发板Lite

```
set_property PACKAGE_PIN R6 [get_ports FMC_HPC_GBTCLK0_M2C_C_P] ## WUJING LITE BOARD
set_property PACKAGE_PIN R5 [get_ports FMC_HPC_GBTCLK0_M2C_C_N]
```

三. 参考设计的系统方案



单路PLIN+pHDMI参考设计数据流

参考设计的数据流如上图所示：

（1）SDI 摄像头输入视频流，帧率为1080p@60Hz。可编程逻辑（PL）端的rx_sdi模块将数据解析为rgb888格式，并复制成两路。其中一路经过预处理后，存入PL DDR，供AI使用；另一路经过预处理后，存入PS DDR，供HDMI显示使用。

（2）PS启动AI，AI完成神经网络运算后，将输出结果写回PL DDR。

（3）PS从PL DDR取回输出结果，完成后处理计算，将输出结果写回PS DDR。

（4）从PS DDR中读取缓存视频数据，通过HDMI输出到显示器。

四. 自定义算子

在FPGA工程中，本参考设计向PL端的custom op提供了两路接口。其中一路为寄存器访问端口，PS作为master端，可对PL端的custom op的寄存器进行读写访问；另一路为数据访问端口，PL端的custom op作为master端，可对PL DDR进行读写访问。

在示例工程中，自定义算子为一个简单的加法器，可对读取到的数据执行加一操作。自定义算子的使用方法可参考以下示例：

（1）PS通过寄存器访问端口，配置指定地址上的寄存器。例如，读访问的数据首地址寄存器，写访问的数据首地址寄存器，访问的数据量寄存器，启动寄存器等。

（2）自定义算子启动运行后，从PL DDR读取输入数据，执行计算，并将输出数据写回PL DDR。自定义算子产生一个结束标志信号。

（3）PS通过寄存器访问端口，轮询结束标志信号寄存器，直至结束。

五. RTL接口

自定义算子的两路接口如下所示：

（1）寄存器访问端口

信号名	位宽	方向	说明
User_Reg_awwaddr	32	输入	写地址
User_Reg_awwdata	32	输入	写值
User_Reg_awwvalid	1	输入	写有效
User_Reg_awwready	1	输出	写就绪
User_Reg_araddr	32	输入	读地址
User_Reg_arvalid	1	输入	读地址有效
User_Reg_arready	1	输出	读地址就绪
User_Reg_rdata	32	输出	读返回数据
User_Reg_rvalid	1	输出	读数据有效
User_Reg_rready	1	输入	读数据就绪

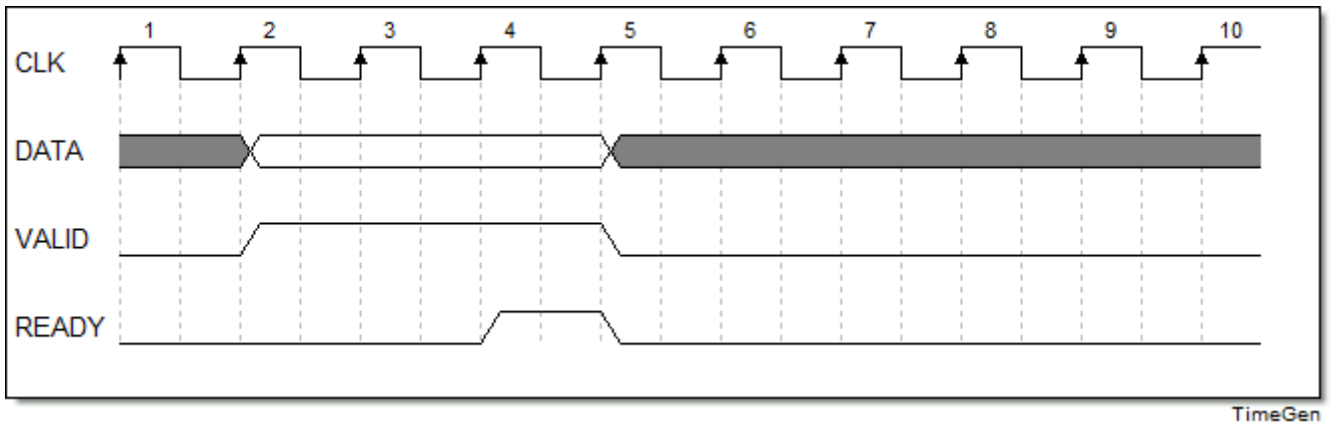
（2）数据访问端口

信号名	位宽	方向	说明
User_Ddr_awwaddr	32	输出	写地址
User_Ddr_awwdata	512	输出	写值
User_Ddr_awwinfo	72	输出	写辅助信息
User_Ddr_awwvalid	1	输出	写有效
User_Ddr_awwready	1	输入	写就绪
User_Ddr_araddr	32	输出	读地址
User_Ddr_arvalid	1	输出	读地址有效
User_Ddr_arinfo	32	输出	读辅助信息
User_Ddr_arready	1	输入	读地址就绪
User_Ddr_rdata	512	输入	读返回数据
User_Ddr_rvalid	1	输入	读数据有效
User_Ddr_rinfo	32	输入	读返回辅助信息
User_Ddr_rready	1	输出	读数据就绪

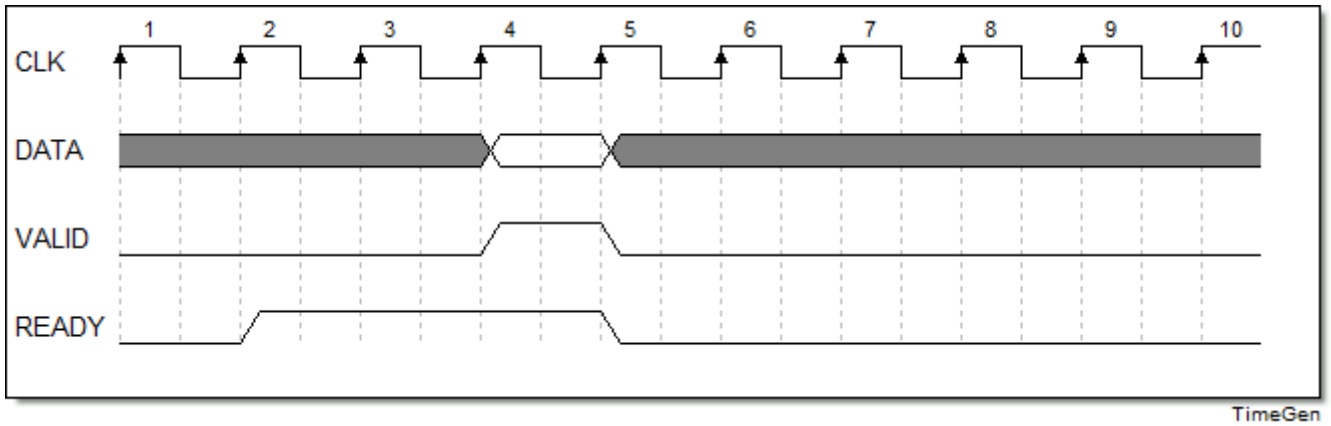
上表中，User_Ddr_awwinfo[70:7]为写操作的字节使能信号，每一比特对应User_Ddr_awwdata信号的一个字节。当User_Ddr_awwinfo[70:7]全为0时，写值全无效；当User_Ddr_awwinfo[70:7]为64'h1时，仅有User_Ddr_awwdata的最低字节有效；当User_Ddr_awwinfo[70:7]全为1时，写值全有效。本demo可以不关心User_Ddr_awwinfo信号的其余比特位以及User_Ddr_arinfo信号的作用。

（3）握手机制

RTL接口采用简单的ready/valid握手机制。master发送数据及valid信号，slave发送ready信号，当valid和ready信号同时有效时，完成一次握手，即一次有效的数据传输。



数据握手时序图 (valid信号先到达)



数据握手时序图 (ready信号先到达)

六. PS程序

6.1 软件工程源文件

```
── deps                                  // 依赖库
── examples
  |── 1_single_input+ai
     |── configs                         // 存放配置文件
     |── imodel                          // 存放网络模型
     |── names                           // 数据集标签
     |── src
        |── adder_demo.cpp               // 加法器demo的源代码
        |── sdicamera+yolov5+hdmi.cpp    // 参考设计的源代码
     |── README.md
     |── CMakeLists.txt
     |── adder_demo                      // 加法器demo的可执行文件
     |── sdicamera+yolov5+hdmi           // 参考设计的可执行文件
```

本DEMO提供加法器作为示例程序，具体可参考examples/1_single_input+ai/PLin+SingleNet+HDMI/src/adder_demo.cpp。

6.2 C程序的API接口

6.2.1 读寄存器

```
int value = device.defaultRegRegion().read(uint32_t address, bool relative);
```

read()函数有两个输入参数：

- @param: address

作用：寄存器的访问地址，32位。custom op的寄存器地址空间为0x400C'0000~0x400F'FFFF。

- @param: relative

作用：访问地址是否为相对地址，默认false。

- @return: value

作用：返回寄存器值，32位。

6.2.2 写寄存器

```
device.defaultRegRegion().write(uint32_t address, uint32_t value, bool relative);
```

write()函数有三个输入参数：

- @param: address

作用：寄存器的写访问地址，32位。custom op的寄存器地址空间为0x400C'0000~0x400F'FFFF。

- @param: value

作用：寄存器的写值，32位。将该值写入对应的寄存器。

- @param: relative

作用：访问地址是否为相对地址，默认false。

6.2.3 分配PL DDR地址空间

```
auto mem_chunk = device.defaultMemRegion().malloc(uint64_t byte_size, bool auto_free,
uint64_t alignment);
```

执行malloc()函数，device将分配一段PL DDR空间供用户使用，通过mem_chunk对该段空间进行读写访问。

malloc()函数有三个输入参数：

- @param: byte_size

作用：分配内存的大小。

- @param: auto_free

作用：指定分配的内存是否会被自动释放。默认false。

- @param: alignment

作用：指定分配的内存地址对齐的字节数。默认64。

6.2.4 写PL DDR

```
mem_chunk.write(uint64_t dest_offset, char* src, uint64_t byte_size)
```

write()函数有三个输入参数：

- @param: dest_offset

作用：数据被写入的偏移位置（相对于mem_chunk的首地址而言）。

- @param: src

作用：char*表示的数据缓存。

- @param: byte_size

作用：写入数据的字节大小。

6.2.5 读PL DDR

```
mem_chunk.read(char* dest, uint64_t src_offset, uint64_t byte_size)
```

read()函数有三个输入参数：

- @param: dest

作用：数据被读取到char*表示的缓存。

- @param: src_offset

作用：从该偏移位置开始读取数据（相对于mem_chunk的首地址而言）。

- @param: byte_size

作用：读取数据的字节大小。